UNIVERSITY OF
Southampton
School of Electronics
and Computer Science

Comp1004: Environments

The Java Library

Coming up

- Recap
  - Encapsulation
  - Constructors
  - Loops
  - Arrays
  - ArrayList
  - Iterators

- The Java Library
  - Implementation vs. interface
  - Example 1: Strings
  - Example 2: Hashmap

Recap

Encapsulation

```
public class Student {

        int age = 20;


        //code omitted

        public static void main(String[] args){
                Student s1 = new Student();
                System.out.println(s1.getAge());
        }

        public int getAge(){
                return age;
        }

}
```

Take this example class where age is modelled as an int

Encapsulation

```
public class Student {

        //int age = 20;
        Calendar dateOfBirth;

        //code omitted

        public static void main(String[] args){
                Student s1 = new Student();
                System.out.println(s1.getAge());
        }

        //public int getAge(){
        //        return age;
        //}

        public int getAge(){
                Calendar rightNow = Calendar.getInstance();
                int a = calculateAge(rightNow, dateofBirth);
                return a;
        }
}
```

Take this example class where age is modelled as an int

We might change the way that age is implemented – e.g. to make it based on the current date.

Because we used an Accessor we do not need to alter main

Encapsulation

```
public class Student {

        //int age = 20;
        protected Calendar dateOfBirth;

        //code omitted

        public static void main(String[] args){
                Student s1 = new Student();
                System.out.println(s1.getAge());
        }

        //public int getAge(){
        //        return age;
        //}

        public int getAge(){
                Calendar rightNow = Calendar.getInstance();
                int a = calculateAge(rightNow, dateofBirth);
                return a;
        }
}
```

The **protected** keyword tells Java that only methods in this class* can access this variable.

*and its sub-classes, but we'll come to that later in the course…

And yes, **public** means the opposite – that all other methods can access it!

Constructors

```
public class Student {

        protected age;

        public Student() {
                age = 20;
        }

        public Student(int a) {
                age = a;
        }

        public static void main(String[] args){
                Student s1 = new Student(19);
                System.out.println(s1.getAge());
        }

        //code omitted
}
```

Constructor Rules:

- Must have the same name as the class
- Does not need a return type
- Can take parameters
- Can be overloaded
- Are invoked at the point of creation using the **new** keyword

Loops

```
int i = 0;
while(i < 10){
        System.out.println(i);
        i++;
}
```

```
int i = 0;
do {
        System.out.println(i);
        i++;
} while (i < 10);
```

```
     initialization; condition; statechange
for (int i = 0;      i < 10;    i++)
{
        System.out.println(i);
}
```

Loops

```
int i = 0;
while(i < 10){
        System.out.println(i);
        i++;
}
```

Condition is checked at start. Loops zero or more times.

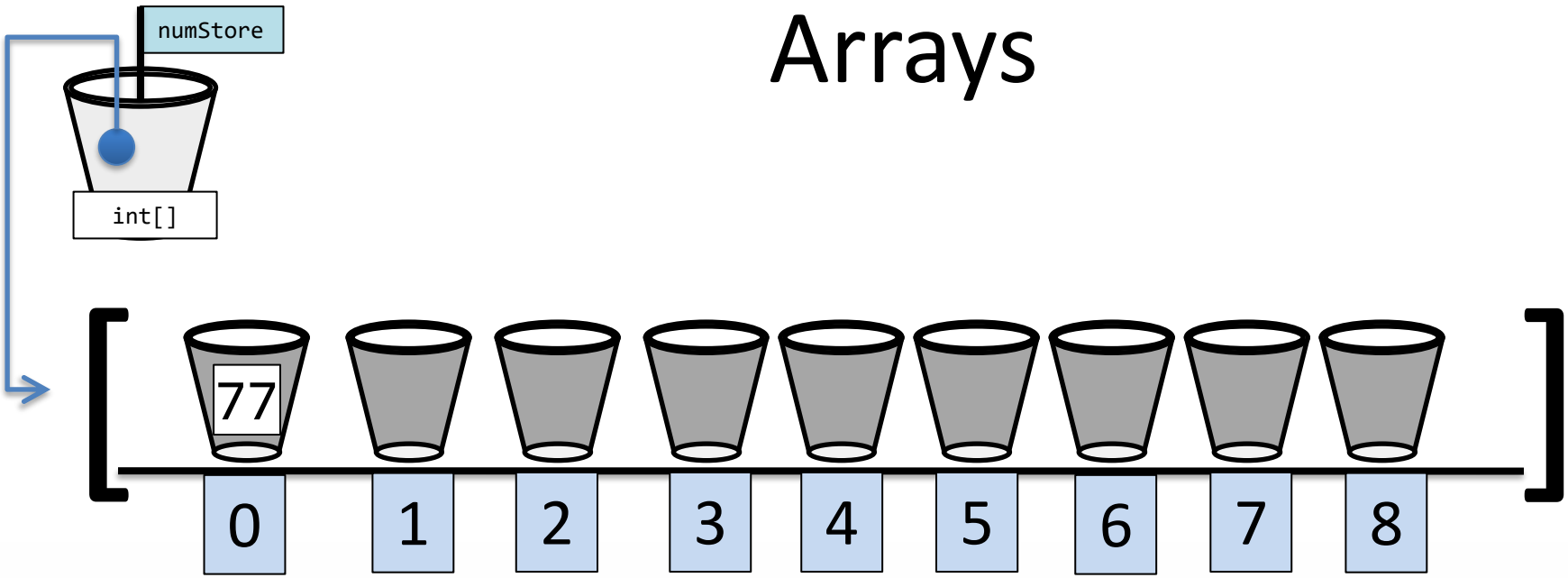
```
int i = 0;
do {
        System.out.println(i);
        i++;
} while (i < 10);
```

Condition is checked at end. Loops one or more times.

```
     initialization; condition; statechange
for (int i = 0;      i < 10;    i++)
{
        System.out.println(i);
}
```

A convenience loop for when we know it advance how many times we want to iterate. Loops zero or more times.

Arrays




```
int[] numStore;
numStore = new int[9];
numStore[0] = 77;
System.out.println(numStore[0]);
```

Declaration

Instantiation

Assignment

Retrieval

Iterating Over an Array

```
int numStore = new int[9];
//some missing code to fill the array with values

for(int i = 0; i < 9; i++){
       System.out.print("Number at position " + i);
       System.out.println(" is " + numStore[i]);
}
```

Iterating over an array is so common that Java now includes a loop specifically to do it.

```
int numStore = new int[9];
//some missing code to fill the array with values

for(int n : numStore){
       System.out.println("Number is " + n);
}
```

Like the for loop the 'for each' loop is a shortcut, that is a bit neater than writing the code the long way.

But it can only be used for access (e.g. n++ would not increment the value in the array)

And it hides the current index

Array vs ArrayList

Array	ArrayList

Declaration Insertion Access Removal

```
Cat[] catArray;
catArray = new Cat[10];

catArray[0] = moggy1;
catArray[1] = moggy2;

callMethodOn(catArray[1]);

catArray[0] = null;
```

```
ArrayList  catAList;
catAList = new ArrayList();

catAList.add(moggy1);
catAList.add(moggy2);

callMethodOn(catAList.get(1));

catAList.remove(moggy1);
```

Generics

```
ArrayList kennel = new ArrayList();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
    kennel.get(i).bark();
}
```

ArrayLists store objects of any type

Which means we can mix up the types of objects in the ArrayList

Which may cause problems later if we make assumptions about what is in there!

In fact this code will not compile, because Java does not know what is in the ArrayList, and therefore will not let you call **bark** on it

Generics

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
kennel.add(new Cat("Mr Tiddles"));


for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}
```

It would be better if we could ensure that the ArrayList **only** contained Dogs in the first place

This is easily done because ArrayList uses a mechanism called **generics**.

We can specify the type allowed when we create the ArrayList.

Now Java will only allow us to add things of type **Dog**. So this line will force a **compile time** error

Iterators

```
ArrayList<Dog> kennel = new ArrayList<Dog>();

kennel.add(new Dog("Rover"));
kennel.add(new Dog("Fido"));
kennel.add(new Dog("Patch"));
```

```
for(int i = 0; i < kennel.size(); i++) {
   kennel.get(i).bark();
}

Iterator<Dog> it = kennel.iterator();

while(it.hasNext()) {
   it.next().bark();
}
```

1) They are neater, and neat code is easier to read and understand

2) They decouple the loop from the collection (notice that in the loop we do not reference the Arraylist at all)

This means we could pass the iterator to a method – and that method does not even need to know what the collection is!

Iterators

```
public void makeThemBark(Iterator<Dog> it) {

   while(it.hasNext()) {
        it.next().bark();
   }

}
```

1) They are neater, and neat code is easier to read and understand

2) They decouple the loop from the collection (notice that in the loop we do not reference the Arraylist at all)

This means we could pass the iterator to a method – and that method does not even need to know what the collection is!

The Java Library

Library

- The java library is full of helpful classes
- Like ArrayList
- What does the inside of an A ook like?
- How does it handle
- How does i e throw an error?
- How do e renumbering when removing element

Implementation

Implementation vs. Interface

- Because of **encapsulation** all we need to know to use the ArrayList class, and the other library classes is what their interface is

- A Class' interface is how we interact with the class
  - It's **public** variables and methods
  - what methods we can call
  - what they do
  - what they will return

Importing

- Library classes must be **imported** using an **import** statement

```
import java.util.ArrayList;

public class myClass{

     ArrayList<String> arrl;
     arrl = new ArrayList<String>;

     public static void main(String[] args){
            //code omitted
     }
}
```

Importing packages

- Classes are organised in **packages**.
- Single classes may be imported:

```
import java.util.ArrayList;
```

- Whole packages can be imported:

```
import java.util.*;
```

Where is the Library?

- All library classes are included in the Java runtime and development environments

- All the documentation is available online:
  - http://download.oracle.com/javase/6/docs/api/

Overview (Java Platform SE 6)

http://download.oracle.com/javase/6/docs/api/

Overview (Java Platform SE 6)

Java™ Platform Standard Ed. 6

All Classes

Packages
java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event

All Classes
AbstractAction
AbstractAnnotationValueVi
AbstractBorder
AbstractButton
AbstractCellEditor
AbstractCollection
AbstractColorChooserPane
AbstractDocument
AbstractDocument.Attribut
AbstractDocument.Conten
AbstractDocument.Elemen
AbstractElementVisitor6
AbstractExecutorService
AbstractInterruptibleChann
AbstractLayoutCache
AbstractLayoutCache.Node
AbstractList
AbstractListModel
AbstractMap
AbstractMap.SimpleEntry
AbstractMap.SimpleImmut
AbstractMarshallerImpl
AbstractMethodError
AbstractOwnableSynchron
AbstractPreferences
AbstractProcessor
AbstractQueue

Overview Package Class Use **Tree** **Deprecated** **Index** **Help**

PREV NEXT **FRAMES** **NO FRAMES**

Java™ Platform Standard Ed. 6

Java™ Platform, Standard Edition 6
API Specification

This document is the API specification for version 6 of the Java™ Platform, Standard Edition.

See:

Description

Packages	
java.applet	Provides the classes necessary to create an applet and the classes an applet uses to communicate with its applet context.
java.awt	Contains all of the classes for creating user interfaces and for painting graphics and images.
java.awt.color	Provides classes for color spaces.
java.awt.datatransfer	Provides interfaces and classes for transferring data between and within applications.
java.awt.dnd	Drag and Drop is a direct manipulation gesture found in many Graphical User Interface systems that provides a mechanism to transfer information between two entities logically associated with presentation elements in the GUI.
java.awt.event	Provides interfaces and classes for dealing with different types of events fired by AWT components.
java.awt.font	Provides classes and interface relating to fonts.
java.awt.geom	Provides the Java 2D classes for defining and performing operations on objects related to two-dimensional geometry.

ArrayList (Java Platform SE 6)

http://download.oracle.com/javase/6/docs/api/

ArrayList (Java Platform SE 6)

Java™ Platform Standard Ed. 6

All Classes

Packages
java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event

ArrayDeque
ArrayIndexOutOfBoundsEx
ArrayList
Arrays
ArrayStoreException
ArrayType
ArrayType
AssertionError
AsyncBoxView
AsyncHandler
AsynchronousCloseExcept
AtomicBoolean
AtomicInteger
AtomicIntegerArray
AtomicIntegerFieldUpdater
AtomicLong
AtomicLongArray
AtomicLongFieldUpdater
AtomicMarkableReference
AtomicReference
AtomicReferenceArray
AtomicReferenceFieldUpd
AtomicStampedReference
AttachmentMarshaller
AttachmentPart
AttachmentUnmarshaller
Attr
Attribute
Attribute

Overview Package Class Use Tree Deprecated Index Help

Java™ Platform Standard Ed. 6

PREV CLASS NEXT CLASS FRAMES NO FRAMES
SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

java.util

Class ArrayList<E>

```
java.lang.Object
  └ java.util.AbstractCollection<E>
      └ java.util.AbstractList<E>
          └ java.util.ArrayList<E>
```

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>, RandomAccess

Direct Known Subclasses:

AttributeList, RoleList, RoleUnresolvedList

```
public class ArrayList<E>
extends AbstractList<E>
implements List<E>, RandomAccess, Cloneable, Serializable
```

Resizable-array implementation of the `List` interface. Implements all optional list operations, and permits all elements, including `null`. In addition to implementing the `List` interface, this class provides methods to manipulate the size of the array that is used internally to store the list. (This class is roughly equivalent to `Vector`, except that it is unsynchronized.)

The `size`, `isEmpty`, `get`, `set`, `iterator`, and `listIterator` operations run in constant time. The `add` operation runs in *amortized constant time*, that is, adding n elements requires O(n) time. All of the other operations run in linear time (roughly speaking). The constant factor is low compared to that for the `LinkedList` implementation.

Each `ArrayList` instance has a *capacity*. The capacity is the size of the array used to store the elements in the list. It is always at least as large as the list size. As elements are added to an ArrayList, its capacity grows automatically. The details of the growth policy are not specified beyond the fact that adding an element has constant amortized time cost.

Images: 3/3 Loaded: 10 KB Speed: 7.36 KB/s Time: 1.360 Done

ArrayList (Java Platform SE 6)

http://download.oracle.com/javase/6/docs/api/

ArrayList (Java Platform SE 6)

Java™ Platform Standard Ed. 6

All Classes

Packages
java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event

ArrayDeque
ArrayIndexOutOfBoundsEx
ArrayList
Arrays
ArrayStoreException
ArrayType
ArrayType
AssertionError
AsyncBoxView
AsyncHandler
AsynchronousCloseExcept
AtomicBoolean
AtomicInteger
AtomicIntegerArray
AtomicIntegerFieldUpdater
AtomicLong
AtomicLongArray
AtomicLongFieldUpdater
AtomicMarkableReference
AtomicReference
AtomicReferenceArray
AtomicReferenceFieldUpd
AtomicStampedReference
AttachmentMarshaller
AttachmentPart
AttachmentUnmarshaller
Attr
Attribute
Attribute

Constructor Summary

ArrayList()	Constructs an empty list with an initial capacity of ten.
ArrayList(Collection<? extends E> c)	Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.
ArrayList(int initialCapacity)	Constructs an empty list with the specified initial capacity.

Method Summary

boolean	**add**(E e) Appends the specified element to the end of this list.
void	**add**(int index, E element) Inserts the specified element at the specified position in this list.
boolean	**addAll**(Collection<? extends E> c) Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator.
boolean	**addAll**(int index, Collection<? extends E> c) Inserts all of the elements in the specified collection into this list, starting at the specified position.
void	**clear**() Removes all of the elements from this list.
Object	**clone**() Returns a shallow copy of this ArrayList instance.
boolean	**contains**(Object o) Returns true if this list contains the specified element.
void	**ensureCapacity**(int minCapacity) Increases the capacity of this ArrayList instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
E	**get**(int index) Returns the element at the specified position in this list.
int	**indexOf**(Object o)

Images: 3/3 Loaded: 10 KB Speed: 7.36 KB/s Time: 1.360 Done

Example 1: Strings

Strings

- Strings are actually objects
  - Did you notice we always use a capital S like other classes?

- You don’t need to import them – they are from the automatically imported from `java.lang.*;`

- As is `System` as in `System.out.println()`

A word about comparing Strings

```
if(input == “hello") {
    //code here
}
```

Tests Identity

```
if(input.equals(“hello")) {
    //code here
}
```

Tests Equality

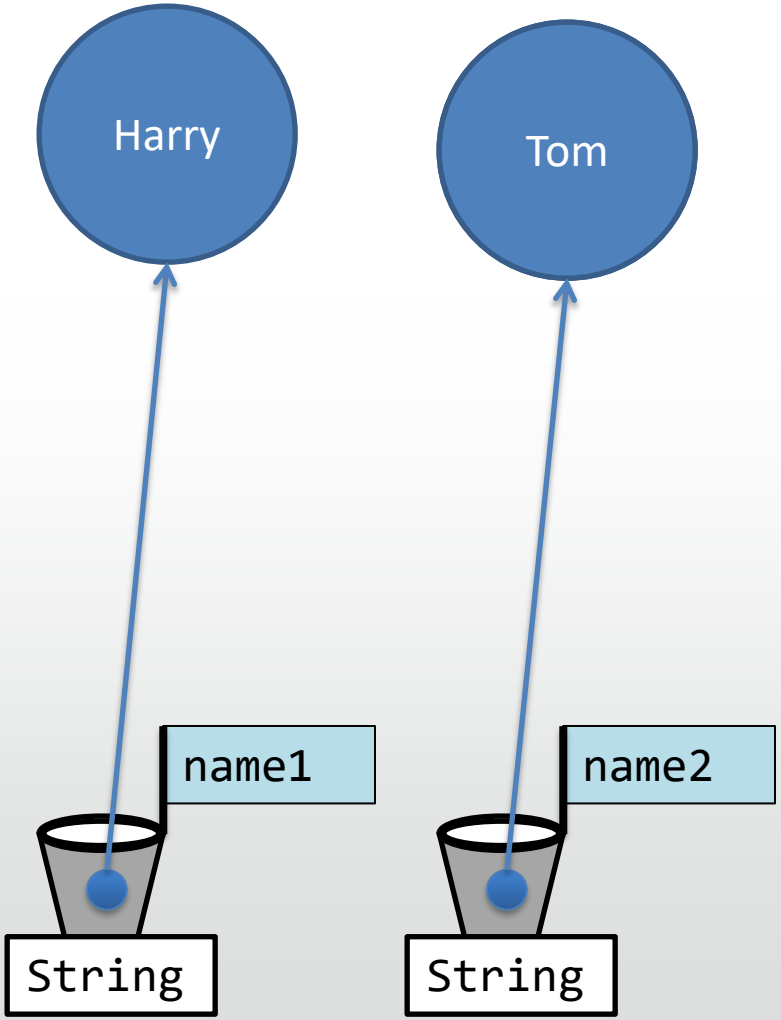
- You probably mean to compare strings using **`.equals`**

identity vs equality

- String name1 = “Harry”;
- String name2 = “Tom”;

- name1 == name2
- False, different **addresses**

- name1.equals(name2)
- False, different **value**

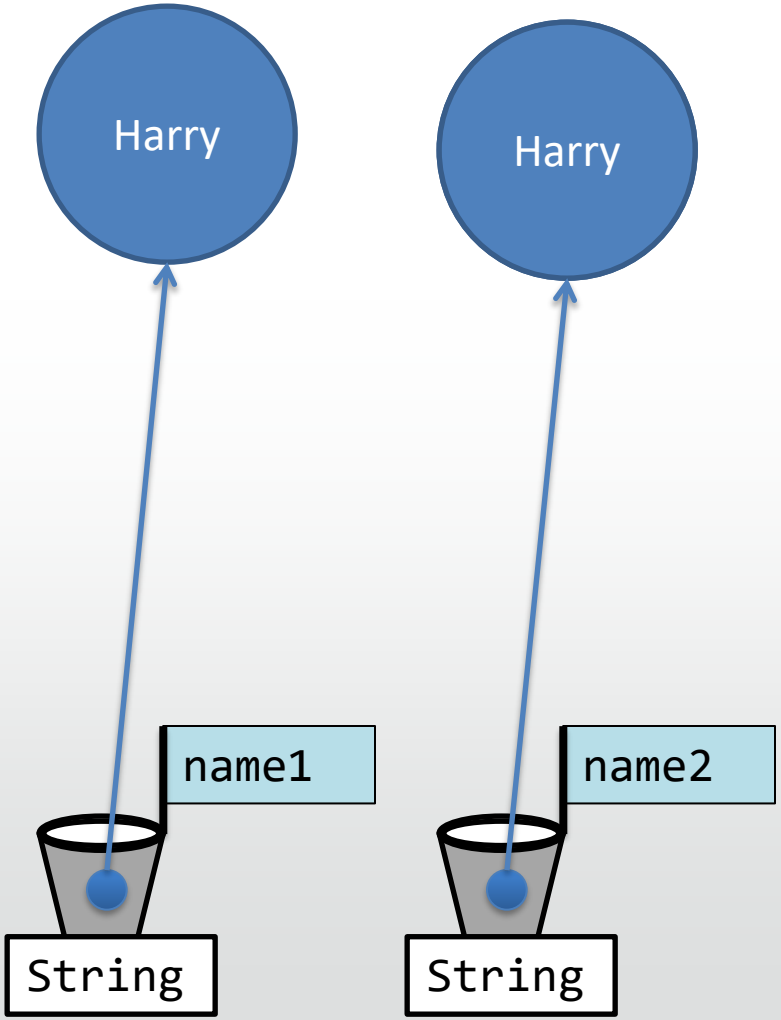


identity vs equality

- String name1 = “Harry”;
- String name2 = “Harry”;

- name1 == name2
- False, different **addresses**

- name1.equals(name2)
- True, same **value**

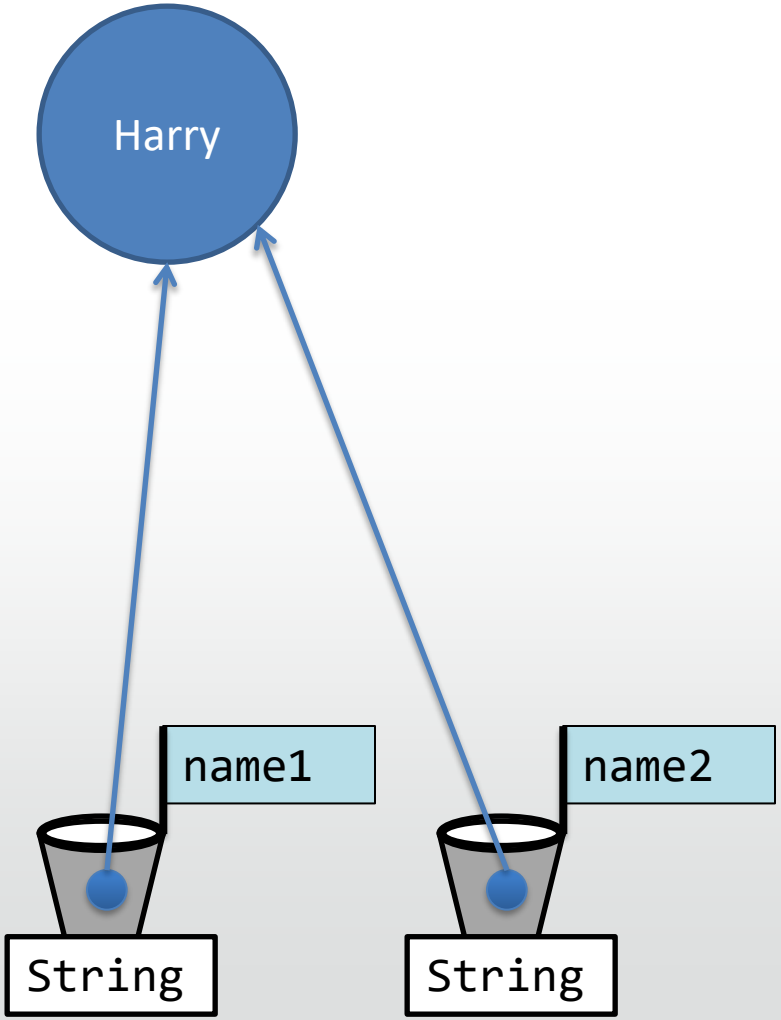


identity vs equality

- String name1 = “Harry”;
- String name2 = name1;

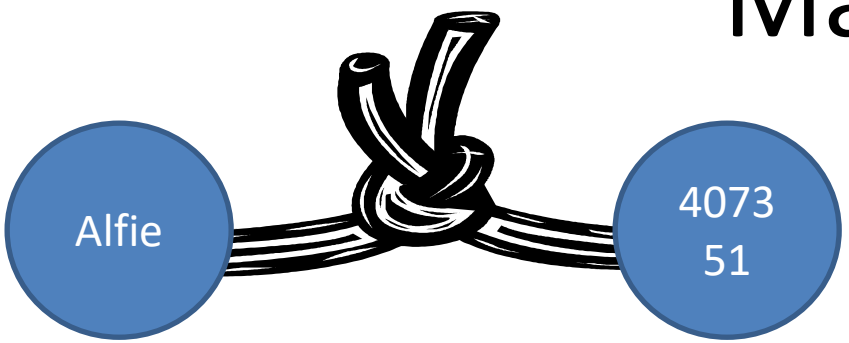
- name1==name2
- True, same **address**

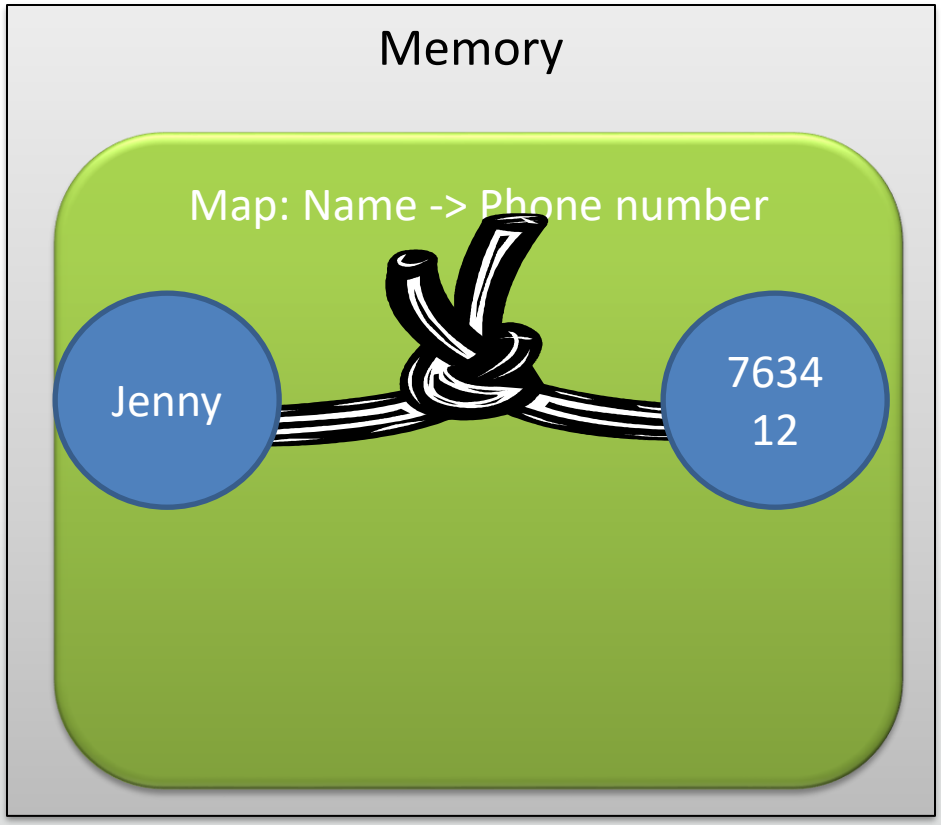
- name1.equals(name2)
- True, same **value**



Example 2: Hashmaps

Maps



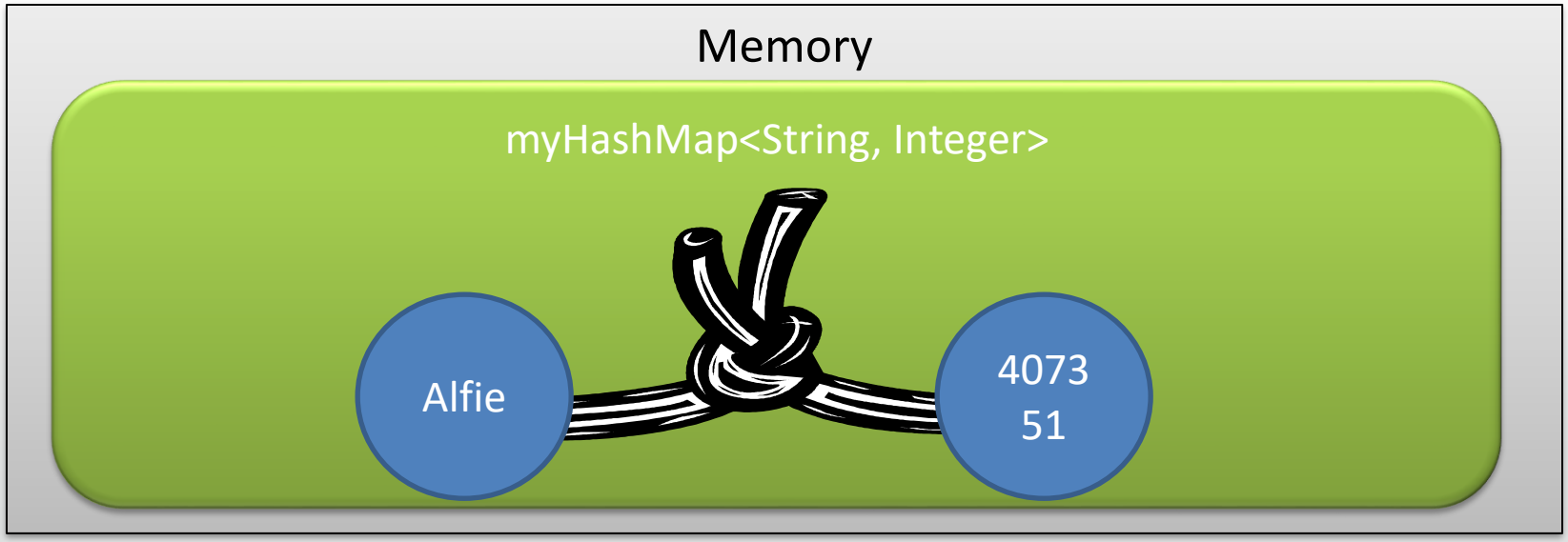


- Maps are a collection type that map a key to a value
- put(“Alfie”,“407351”);
- and so on

Lookup

- Lookup is the act of supplying the key and having the value returned

```
String num = myHashMap.get("Alfie");
```



Bringing it together…

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

Bringing it together…

What is this?

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

Bringing it together…

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

What is this?

HashMap is a **generic class**, this means we should tell it what two types it maps together

Bringing it together…

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

What is this?

HashMap is a **generic class**, this means we should tell it what two types it maps together

What is happening here?

Bringing it together…

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

What is this?

HashMap is a **generic class**, this means we should tell it what two types it maps together

What is happening here?

This is **autoboxing** – the ints are automatically turned into Integers for us

Bringing it together…

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

What is this?

HashMap is a **generic class**, this means we should tell it what two types it maps together

What is happening here?

This is **autoboxing** – the ints are automatically turned into Integers for us

Which type of String comparison is being used?

Bringing it together…

```
import java.util.HashMap;

//code omitted

HashMap<String, Integer> marks;
marks = new HashMap<String, Integer>();

marks.put("Alice", 75);
marks.put("Bob", 62);
marks.put("Colin", 68);

System.out.println("Bob got " + marks.get("Bob"));
```

What is this?

HashMap is a **generic class**, this means we should tell it what two types it maps together

What is happening here?

This is **autoboxing** – the ints are automatically turned into Integers for us

Which type of String comparison is being used?

Equality (not Identity). It is using the **.equals** method

Summary

- Recap
  - Encapsulation
  - Constructors
  - Loops
  - Arrays
  - ArrayList
  - Iterators

- The Java Library
  - Class interface
  - Example 1: Strings
  - Example 2: Hashmap